Scenario 1: Bad Container Image Deployment

Objective

Trace an incomplete Kubernetes rollout from a Datadog signal to the Deployment image and Amazon ECR, then recover safely.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/bad-deploy.sh banking banking-backend banking-backend
```

The script changes the live Deployment image to a nonexistent tag. Wait for Datadog and Kubernetes to observe the failed rollout before troubleshooting. Do not run another scenario until this one has been recovered.

Situation

The banking application still serves users, but a new release has not completed. Do not run the trigger yourself unless you are facilitating the lab.

Symptoms

- The rollout does not complete.
- New pods do not start while existing pods may remain healthy.

What You Know

- The issue began immediately after a banking backend release.
- Datadog shows the `banking-backend` Deployment with unavailable replicas and new pods that are not Ready.
- Existing pods may still serve traffic.

Start With Datadog

1. Go to **Infrastructure > Kubernetes > Explorer**, select **Pods** under **Select Resources**, and set the time range to **Past 30 Minutes**.

2. In **Filter by**, enter `kube_namespace:banking kube_deployment:banking-backend`. Open the newest pod whose status is `ErrImagePull` or `ImagePullBackOff`, then record its pod name and image.
3. Go to **Events > Explorer**, keep **Past 30 Minutes**, and search `kube_namespace:banking status:(warning OR error)`. Open an event containing `ErrImagePull`, `ImagePullBackOff`, `Failed to pull image`, or `does-not-exist`.

`kubernetes_state.deployment.replicas_unavailable` can remain `0` in this scenario because the two old pods continue serving while Kubernetes creates one bad surge pod. Use the desired-versus-updated gap plus the image-pull event as the primary evidence.

Record the first event timestamp, failed pod, and requested image before using `kubectl`.

Troubleshooting

1. Check the Deployment summary.

```
kubectl get deployment banking-backend -n banking
```

Expected output: `READY` remains `2/2`, `AVAILABLE` remains `2`, but `UP-TO-DATE` is only `1`. This means the old replicas are healthy while the new rollout is incomplete.

2. Find the failed new pod.

```
kubectl get pods -n banking -l app=banking-backend
```

Expected output: two older pods are `Running`, while the newest pod is `ErrImagePull` or `ImagePullBackOff` and shows `0/1` Ready.

3. Inspect the failed pod and its recent events.

```
kubectl describe pod <pod-name> -n banking
```

Expected output: the **Events** section contains `Failed to pull image`, the requested tag ends in `does-not-exist`, and the registry reports that the image was not found.

4. Confirm the image configured on the Deployment.

```
kubectl get deployment banking-backend -n banking \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Expected output: an ECR image URL ending in `:does-not-exist`.

5. Compare the requested tag with the images that actually exist in ECR.

```
aws ecr describe-images --repository-name sre-lab/banking-backend \
  --region us-east-1 --query 'imageDetails[].imageTags' --output table
```

Expected output: valid image tags are listed, but `does-not-exist` is absent. The missing ECR tag is the root cause.

Important Clues

Connect the Datadog replica mismatch with `ErrImagePull` or `ImagePullBackOff`, Kubernetes events, and the ECR image list.

Root Cause

Record your conclusion from Datadog, Kubernetes, and ECR evidence. Confirm it with the instructor after the exercise.

Fix

Roll back to the last working revision.

```
kubectl rollout history deployment/banking-backend -n banking
kubectl rollout undo deployment/banking-backend -n banking
```

Validation

```
kubectl rollout status deployment/banking-backend -n banking
kubectl get pods -n banking
```

Confirm the application still works, unavailable replicas return to zero, and the Datadog monitor recovers.

DevOps Lesson

Validate an ECR tag before changing a Kubernetes Deployment. A healthy old ReplicaSet does not mean a release succeeded.

STAR Method

Situation

Summarize the failed rollout without naming the cause first.

Task

Explain the safe recovery objective.

Action

Describe the evidence path from Datadog to Kubernetes and ECR.

Result

State how rollout and observability recovery were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our EKS releases referenced image tag `v2.4.1`, but only `v2.4.0` existed in ECR. My task was to restore a safe rollout without interrupting the old healthy replicas. I used Datadog and Kubernetes events to confirm `ImagePullBackOff`, verified that the tag was missing in ECR, and rolled back the Deployment. The application remained available, the rollout recovered, and we added image-tag validation to the pipeline.

Scenario 2: Bad ConfigMap Rollout

Objective

Diagnose a rollout where application configuration and Kubernetes health probes no longer agree.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-config.sh student-portal
```

The script saves the current port, changes the ConfigMap, and restarts the student portal backend Deployment. Wait for Datadog and Kubernetes to observe the rollout failure before troubleshooting.

Situation

The student portal has reduced rollout capacity after a configuration release. Existing pods may still serve requests, but new pods do not become Ready.

Symptoms

- New pods fail to become Ready.
- Restart counts may increase while older pods still serve.

What You Know

- Datadog shows unavailable replicas and increasing container restarts for `student-portal-backend`.
- Kubernetes health events began after a rollout.
- The process in each new container appears to start.

Start With Datadog

1. Go to **Infrastructure > Kubernetes > Explorer**, select **Pods**, set **Past 30 Minutes**, and enter `kube_namespace:student-portal kube_deployment:student-portal-backend` in **Filter by**. Compare Ready status and restarts on the old and new pods.
2. Go to **Events > Explorer** and search `kube_namespace:student-portal status:(warning OR error)`. Open the readiness-probe, liveness-probe, or BackOff event for the newest backend pod.
3. Go to **Logs > Explorer**, set **Past 30 Minutes**, and search `service:student-portal-backend`. Open a startup log and record the port on which the process is listening.

Record the newest pod, first probe-event timestamp, restart change, and logged listening port.

Troubleshooting

1. Check the rollout and identify the unhealthy new pod.

```
kubectl get deployment student-portal-backend -n student-portal
kubectl get pods -n student-portal -l app=student-portal-backend
```

Expected output: the rollout is incomplete, and the newest pod is not Ready or is restarting while older pods remain Running.

2. Inspect the new pod.

```
kubectl describe pod <pod-name> -n student-portal
```

Expected output: events show readiness or liveness probe failures against port `4000`, often followed by `BackOff` or a container restart.

3. Read the application startup logs.

```
kubectl logs <pod-name> -n student-portal
```

Expected output: the application reports that it is listening on port `4001`. This conflicts with the probes checking port `4000`.

4. Compare the ConfigMap value with the Deployment probes.

```
kubectl get configmap student-portal-backend-config -n student-portal -o yaml
kubectl get deployment student-portal-backend -n student-portal \
  -o jsonpath='{.spec.template.spec.containers[0].readinessProbe.httpGet.port}{"\n"}'
```

Expected output: the ConfigMap contains `PORT: "4001"`, but the readiness probe returns `4000`. The port mismatch is the root cause.

Important Clues

Connect the probe failures to the application listening port and the port checked by the readiness and liveness probes.

Root Cause

Record the configuration and probe mismatch supported by your evidence. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-config.sh student-portal --undo
```

The script restores the saved `PORT` value and restarts the Deployment.

Validation

```
kubectl rollout status deployment/student-portal-backend -n student-portal
kubectl get pods -n student-portal
```

Confirm every pod is Ready, restarts stop increasing, the app works, and both Datadog monitors recover.

DevOps Lesson

Validate configuration and health-probe contracts together before rollout.

STAR Method

Situation

Summarize the partial-capacity rollout.

Task

Explain why only new pods needed investigation.

Action

Describe the Datadog, event, log, ConfigMap, and probe comparison.

Result

State how readiness and monitor recovery were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our internal applications stopped bringing new pods to Ready after a configuration rollout. My task was to identify why only the new pods failed. Datadog events and logs showed probe failures and an application listening on port 4001, while Kubernetes probes still checked port 4000. I restored the correct ConfigMap value and restarted the Deployment. All replicas became Ready, and configuration-to-probe validation was added to deployment checks.

Scenario 3: OOMKilled and Kubernetes Resource Limits

Objective

Use Datadog and Kubernetes evidence to distinguish memory pressure from an unsafe container resource limit.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/shrink-limits.sh support-tickets
```

The script saves the current resources and applies a deliberately unsafe memory request and limit. Wait for restarts and Datadog signals before troubleshooting.

Situation

The support tickets service is unstable. Backend pods restart repeatedly while some requests still succeed.

Symptoms

- Backend pods restart repeatedly.
- Application stability degrades.

What You Know

- Datadog shows backend memory near its configured ceiling.
- Container restart counts are increasing.
- The problem began after a workload resource change.

Start With Datadog

1. Go to **Dashboards > SRE Lab Scenario Signals** and set the time range to **Past 30 Minutes**.
2. In **Memory Usage and Limits**, filter `kube_namespace:support-tickets` and `kube_deployment:support-tickets-backend`. Compare `kubernetes.memory.usage` with `kubernetes.memory.limits` immediately before each restart.
3. In **Container Restarts**, confirm `kubernetes.containers.restarts` increases for the same Deployment.

Record the memory value, configured limit, restart timestamp, pod name, and monitor transition. Confirm the exact `OOMKilled` reason later with `kubectl describe`.

Troubleshooting

1. Identify the restarting pod.

```
kubectl get pods -n support-tickets -l app=support-tickets-backend
```

Expected output: the backend pod is Running again, but its `RESTARTS` count is greater than zero and continues increasing.

2. Inspect the previous container termination.

```
kubectl describe pod <pod-name> -n support-tickets
```

Expected output: under **Last State**, the reason is `OOMKilled` and the exit code is `137`.

3. Compare live memory with the configured limit.

```
kubectl top pod <pod-name> -n support-tickets
kubectl get deployment support-tickets-backend -n support-tickets \
  -o jsonpath='{.spec.template.spec.containers[0].resources}{"\n"}'
```

Expected output: memory usage approaches the deliberately reduced `20Mi` memory limit. The container exceeds that limit and Kubernetes terminates it.

4. Check the previous container logs for supporting evidence.

```
kubectl logs <pod-name> -n support-tickets --previous
```

Expected output: logs stop around the restart time and may not contain a graceful application error. `OOMKilled` in the pod state is the authoritative evidence.

Important Clues

Look for `Reason: OOMKilled`. Compare observed memory consumption with `resources.requests.memory` and `resources.limits.memory`.

Root Cause

Record the relationship between observed memory, the previous container state, and the configured limit. Confirm it with the instructor.

Fix

Restore the known baseline, then use observed utilization to propose a justified long-term request and limit.

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
kubectl rollout status deployment/support-tickets-backend -n support-tickets
```

Do not remove the limit or choose an arbitrary large value.

Validation

Confirm pods remain Ready, restart counts stop increasing, memory stays below the limit, requests succeed, and both Datadog monitors recover.

DevOps Lesson

Resource limits must reflect measured workload behavior. Admission control can accept a value that is operationally unsafe.

STAR Method

Situation

Summarize the unstable workload.

Task

Explain how you separated application, Kubernetes, and infrastructure causes.

Action

Describe the memory, restart, previous-state, and resource comparison.

Result

State how stability and memory headroom were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

An internal backend repeatedly restarted after its memory limit was reduced to 20 MiB. My task was to distinguish an application failure from a Kubernetes resource problem. I correlated Datadog memory and

restart signals with exit code 137 and `OOMKilled` in the pod state, then restored a measured memory limit. The restart loop stopped, pods remained Ready, and future resource values were based on observed usage.

Scenario 4: ALB and Kubernetes Ingress Routing Failure

Objective

Trace HTTP 404 responses across Datadog, Route 53, ALB, Ingress, Service, endpoints, and pods.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-ingress.sh food-delivery
```

The script saves the current Ingress backend and changes the live route. Allow one or two minutes for the AWS Load Balancer Controller to reconcile before testing the public URL.

Situation

Users receive HTTP 404 from the food delivery home page even though the Kubernetes workloads report healthy.

Symptoms

- The public hostname returns HTTP 404.
- Kubernetes workloads remain Running and Ready.

What You Know

- `food-delivery.<lab-domain>` resolves and the ALB accepts connections.
- Pods are Running and Ready.
- Datadog Trace Explorer shows requests to `service:food-delivery-backend` with `http.status_code:404` and resource `GET /`.

Start With Datadog

1. Go to **Dashboards > SRE Lab Scenario Signals** and set the time range to **Past 30 Minutes**.
2. Inspect **Backend GET / HTTP 404 Responses** for `service:food-delivery-backend`. This widget uses `trace.express.request.hits.by_http_status` filtered by `http.status_code:404` and metric-normalized `resource_name:get_/`.

3. Go to **Monitors > Manage Monitors**, search `[SRE Lab] Unexpected backend root traffic`, open it, and expand the `food-delivery-backend` group.

This repository does not install the Datadog AWS integration, so ALB listeners, rules, and target health are not available in Datadog. Record the 404 trace timestamp and backend pod, then continue with AWS CLI and Kubernetes.

Troubleshooting

1. Confirm that the workloads are healthy.

```
kubectl get pods -n food-delivery
```

Expected output: frontend and backend pods are Running and Ready. This moves the investigation away from pod health and toward request routing.

2. Inspect the public Ingress route.

```
kubectl describe ingress food-delivery -n food-delivery
```

Expected output: the `/` path points to `food-delivery-backend:4000`. It should point to `food-delivery-frontend:80`.

3. Confirm that both Services and endpoint sets exist.

```
kubectl get svc,endpoints -n food-delivery
```

Expected output: the frontend Service listens on port `80`, the backend listens on `4000`, and both have endpoints. This proves the failure is an incorrect Ingress destination rather than a missing Service or pod.

4. Confirm the ALB is present and serving the incorrect rule.

```
aws elbv2 describe-load-balancers --region us-east-1
aws elbv2 describe-listeners --load-balancer-arn <alb-arn> --region us-east-1
aws elbv2 describe-rules --listener-arn <listener-arn> --region us-east-1
```

Expected output: the ALB and HTTPS listener are active, and the host rule forwards traffic to the target group created from the incorrect backend Service. The Ingress backend selection is the root cause.

Important Clues

Follow DNS to ALB, listener, target group, Ingress, Service, endpoints, pod, and application. Determine why the backend answers a request intended for the frontend.

Root Cause

Record which routing hop sends traffic to the wrong destination. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-ingress.sh food-delivery --undo
kubectl describe ingress food-delivery -n food-delivery
```

The source-of-truth manifest is `ingress/food-delivery-ingress.yaml`, whose backend is `food-delivery-frontend:80`.

Validation

Test `https://food-delivery.<lab-domain>/`, confirm the controller reconciles the Ingress, and verify the backend `GET /` trace rate returns to zero in Datadog.

DevOps Lesson

Healthy pods do not guarantee healthy user traffic. Validate the whole routing path.

STAR Method

Situation

Summarize the user-visible 404 with healthy workloads.

Task

Explain the need to locate the failing routing hop.

Action

Describe the Datadog, AWS, and Kubernetes traffic-path checks.

Result

State how external access and trace recovery were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users received HTTP 404 responses from one of our applications even though every pod was healthy. My task was to locate the failing hop in the public request path. Datadog showed `GET /` reaching the backend, and the Ingress revealed that `/` targeted port 4000 instead of the frontend on port 80. I corrected the route, waited for ALB reconciliation, and confirmed successful external requests and normal traces.

Scenario 5: High Application Latency Using Datadog

Objective

Use Datadog APM to identify a slow workload before investigating Kubernetes and applying a focused recovery.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/inject-latency.sh ecommerce 3000
```

The request reaches one backend pod because chaos state is stored per process. Generate application traffic and wait for APM ingestion before troubleshooting. Repeat through the documented port-forward method only when you want every backend pod affected.

Situation

Users report that ecommerce pages load slowly. Do not restart pods before scoping the problem.

Symptoms

- Ecommerce responses are intermittently slow.
- Pods remain Ready.

What You Know

- Datadog reports elevated p95 latency.
- The issue affects `ecommerce-backend` while pods remain Ready.
- The start time does not match an AWS or database change.

Start With Datadog

1. Go to **Dashboards > Dashboard List**, open **ecommerce**, and set the time range to **Past 30 Minutes**. In **p95 Latency**, look for `p95:trace.express.request{env:lab,service:ecommerce-backend}` rising above the normal baseline.
2. Go to **Dashboards > Dashboard List**, open **SRE Lab Scenario Signals**, and compare **Backend p95 Latency** with memory, restarts, and available replicas. The scenario should raise latency without

requiring resource saturation or unhealthy pods.

3. Go to **Monitors > Manage Monitors**, search `[SRE Lab] High p95 latency`, open it, and expand the `ecommerce-backend` group. Record the alert start time and value.

The key scope is slow APM traces for one service while Kubernetes health and resource signals remain near baseline.

Troubleshooting

1. Confirm Kubernetes health.

```
kubectl get deployment,pods -n ecommerce
```

Expected output: the Deployment is fully available, pods are Ready, and restart counts remain stable. The application is slow but not unavailable.

2. Check resource usage.

```
kubectl top pods -n ecommerce
```

Expected output: CPU and memory remain near their normal baseline. This makes resource saturation and OOM restarts unlikely.

3. Inspect the backend logs during the slow-trace timestamps.

```
kubectl logs deployment/ecommerce-backend -n ecommerce --since=15m
```

Expected output: requests complete without crashes, but response timing corresponds to the slow traces. There should be no probe failure, OOM, or restart pattern.

4. Check the application's current chaos state.

```
curl -s "https://ecommerce.$(cat .lab-domain)/api/chaos"
```

Expected output: JSON containing `"latencyMs":3000`. This in-memory setting explains traces taking approximately three seconds while Kubernetes remains healthy.

Important Clues

Determine why one or more backend containers add time before every response. Use trace duration and the chaos-state endpoint as evidence.

```
curl -s https://ecommerce.$(cat .lab-domain)/api/chaos
```

Root Cause

Record the workload behavior that accounts for the trace duration while Kubernetes resource metrics remain normal. Confirm it with the instructor.

Fix

Clear the workload's injected latency. Chaos state is per process, so repeat until every backend pod is reset, or reset each pod through port-forwarding as documented in the README.

```
./scripts/chaos/reset.sh ecommerce
```

Validation

Generate normal requests. Confirm p95 latency and traces return to baseline, pods remain healthy, Kubernetes metrics remain normal, and the Datadog monitor recovers.

DevOps Lesson

Use traces to isolate the slow workload before taking action. A restart may hide evidence without explaining the cause.

STAR Method

Situation

Summarize the latency report and initial scope.

Task

Explain the need to identify the affected workload before acting.

Action

Describe the dashboard, trace, Kubernetes, log, and workload-state evidence.

Result

State how normal latency and healthy pods were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users reported slow responses from one of our applications while its EKS pods remained healthy. My task was to isolate the source before restarting anything. Datadog APM showed requests taking approximately three seconds, while CPU, memory, readiness, and restarts stayed normal. I found and cleared a 3000 ms application delay setting, after which p95 latency returned to baseline and the monitor recovered.